ARQUITETURA DE DADOS

DO DATA WAREHOUSE AO DATA LAKEHOUSE

Prof. Gustavo Dias

No cenário atual, em que os dados assumem papel central na tomada de decisão, compreender as diferentes abordagens de arquitetura de dados tornou-se essencial para profissionais da área. Este material de estudo tem como base o livro Deciphering Data Architectures, de James Serra, especialista em soluções de dados da Microsoft, e se propõe a guiar o leitor na análise e escolha entre modelos como Modern Data Warehouse, Data Fabric, Data Lakehouse e Data Mesh.

Ao longo do conteúdo, será apresentada a evolução das arquiteturas de dados — desde os tradicionais bancos de dados relacionais até as modernas abordagens de Big Data — com foco nos principais desafios relacionados à integração, escalabilidade e governança. A partir de uma perspectiva neutra em relação a fornecedores, o material explora vantagens, limitações e contextos de aplicação de cada modelo, contribuindo para decisões mais assertivas e evitando erros comuns em projetos de dados.

Mais do que um panorama conceitual, este estudo busca capacitar o leitor a transformar dados brutos em ativos estratégicos, por meio de uma infraestrutura eficiente e orientada a resultados. Por fim, destaca-se a importância do autoatendimento analítico e do avanço na maturidade digital como pilares fundamentais para o sucesso das organizações na era orientada por dados.

1. Da Explosão de Dados à Transformação Organizacional

No contexto da transformação digital, o crescimento exponencial do volume de dados tem impulsionado as organizações a repensarem não apenas suas tecnologias, mas também seus processos, cultura e capacidade analítica. Nesse cenário, surge o conceito de Big Data, que vai além da simples ideia de grandes volumes de informação e passa a ser compreendido por meio de múltiplas dimensões que refletem sua complexidade e potencial estratégico.

Essas dimensões são conhecidas como os “6 Vs” do Big Data, e ajudam a caracterizar os desafios e oportunidades no uso de dados. O volume refere-se à enorme quantidade de dados gerados continuamente, provenientes de diversas fontes como redes sociais, transações digitais e dispositivos conectados. Já a variedade diz respeito à diversidade de formatos, que incluem dados estruturados (como bancos relacionais), semiestruturados (JSON, XML, logs), não estruturados (textos, e-mails, PDFs) e até conteúdos binários, como imagens, áudios e vídeos.

A velocidade está relacionada à rapidez com que esses dados são gerados e precisam ser processados — variando desde cargas em lote até fluxos em tempo real. A veracidade destaca a necessidade de garantir a qualidade e confiabilidade dessas informações, especialmente considerando a heterogeneidade das fontes. Complementarmente, a variabilidade aborda a inconsistência nos padrões e comportamentos dos dados ao longo do tempo, exigindo abordagens flexíveis de processamento e análise. Por fim, o valor representa o objetivo central de todo esse ecossistema: transformar dados em insights relevantes que apoiem decisões e gerem vantagens competitivas.

Diante desse cenário, as organizações percorrem uma jornada evolutiva em sua capacidade de lidar com dados, conhecida como maturidade analítica. No estágio inicial, chamado reativo, os dados encontram-se dispersos em planilhas e sistemas isolados, com baixa governança e pouca confiabilidade — um ambiente frequentemente marcado pelos chamados spreadmarts. À medida que evoluem para o estágio informativo, as empresas passam a centralizar seus dados, viabilizando análises mais consistentes, embora ainda com foco no passado e com limitações de escalabilidade e atualização.

O avanço para o estágio preditivo representa uma mudança significativa, com a adoção de tecnologias mais robustas, frequentemente baseadas em nuvem, capazes de lidar com grandes volumes e diferentes tipos de dados em alta frequência. Nesse nível, a análise deixa de ser apenas descritiva e passa a antecipar cenários, utilizando técnicas como machine learning e processamento em tempo real para apoiar decisões mais assertivas.

Por fim, no estágio transformativo, os dados tornam-se um ativo estratégico plenamente integrado à organização. A arquitetura é flexível, escalável e preparada para lidar com qualquer tipo de dado, enquanto ferramentas de autoatendimento permitem que usuários de diferentes áreas explorem informações de forma autônoma. Nesse ponto, a empresa não apenas utiliza dados — ela é orientada por dados.

Essa evolução reflete, portanto, a própria transformação das organizações na era digital: uma transição de ambientes fragmentados e reativos para ecossistemas integrados, inteligentes e orientados à geração contínua de valor.

2. Data Warehouse x Data Lake

Ao longo das aulas anteriores, exploramos os fundamentos do Data Warehouse, compreendendo sua importância na organização, padronização e análise de dados estruturados para suporte à tomada de decisão. No entanto, com o avanço do Big Data e o aumento expressivo no volume, variedade e velocidade das informações, novas abordagens arquiteturais passaram a ganhar espaço — entre elas, o Data Lake, que surge como uma alternativa mais flexível e escalável para o armazenamento de dados. Nesse contexto, torna-se essencial entender não apenas o que diferencia essas duas abordagens, mas também como elas se complementam no cenário atual. A seguir, serão apresentadas as principais diferenças entre Data Lake e Data Warehouse, destacando aspectos como estrutura, modelagem, tipos de dados, perfis de usuários e custos, fundamentais para a escolha da arquitetura mais adequada às necessidades organizacionais.

As principais diferenças entre um Data Lake e um Data Warehouse residem na forma como os dados são estruturados, na flexibilidade do esquema, nos tipos de dados suportados e no perfil dos usuários que os acessam.

- Estrutura e Armazenamento: Um Data Warehouse (RDW) utiliza armazenamento relacional, organizando os dados em tabelas com linhas e colunas bem definidas
  Já um Data Lake é essencialmente um sistema de arquivos que utiliza armazenamento de objetos, onde os dados não precisam de uma estrutura inicial para serem guardados.
- Abordagem do Esquema (Schema):
  O Data Warehouse segue o modelo schema-on-write (esquema na escrita), o que exige que a estrutura, tabelas e campos sejam criados e os dados limpos antes de serem carregados
  O Data Lake utiliza o modelo schema-on-read (esquema na leitura), permitindo que os dados sejam ingeridos em seu formato bruto (natural). O esquema só é definido quando os dados são efetivamente acessados ou consumidos

- Tipos de Dados: Enquanto o Data Warehouse é projetado especificamente para dados estruturados provenientes de sistemas operacionais, o Data Lake é capaz de lidar com uma enorme variedade de formatos, incluindo dados estruturados, semiestruturados (como JSON, XML e logs) e não estruturados (como e-mails, documentos e arquivos de mídia).
- Usuários e Complexidade: O Data Warehouse é a ferramenta preferida de usuários de negócios e analistas que precisam de relatórios e dashboards de fácil compreensão, baseados em uma "versão única da verdade"
  O Data Lake, por ser mais complexo de consultar, atende melhor a cientistas de dados e usuários avançados que possuem habilidades técnicas para explorar dados brutos usando ferramentas como Python ou Spark.
- Custo e Escalabilidade: Armazenar dados em um Data Lake é consideravelmente mais barato do que em um Data Warehouse, o que permite que as empresas coletem dados "por precaução" para uso futuro
  O Data Warehouse, embora ofereça performance superior para consultas analíticas complexas, envolve um custo total de solução mais elevado

Pode-se dizer, portanto, que o Data Warehouse foca na análise histórica e de tendências com dados já processados, enquanto o Data Lake foca no armazenamento massivo de dados brutos para exploração rápida e treinamento de modelos de machine learning

No cenário atual, muitas empresas utilizam a arquitetura de Modern Data Warehouse (MDW), que combina ambos para aproveitar o melhor de cada mundo

2.1 Schema On-Read

O conceito de schema-on-read (esquema na leitura) refere-se a uma abordagem de gerenciamento de dados em que o esquema ou a estrutura formal dos dados é aplicado apenas no momento em que as informações são lidas ou acessadas, em vez de ser definido no momento em que os dados são gravados.

De acordo com as fontes, as principais características dessa abordagem são:

- Ingestão sem Estrutura Prévia: Os dados podem ser carregados no sistema de armazenamento sem precisar estar em conformidade com um esquema rígido inicial.
  Isso torna o processo de ingestão extremamente rápido e simples, muitas vezes resumindo-se a apenas copiar arquivos para um repositório.

- Flexibilidade de Formatos: Esta técnica oferece maior flexibilidade para lidar com dados não estruturados ou semiestruturados (como JSON, XML e logs) em seu formato natural ou bruto.
- Definição no Consumo: A estrutura, os tipos de dados e os relacionamentos só precisam ser definidos quando os dados são efetivamente consultados ou consumidos por uma aplicação ou usuário.

  Como os arquivos em um repositório bruto podem não conter contexto, o esquema é criado ou extraído de arquivos separados no momento da leitura para fornecer o contexto necessário.

- Agilidade na Exploração: Como exige pouco trabalho inicial, o schema-on-read permite que cientistas de dados e usuários avançados explorem os dados rapidamente para descobrir quais perguntas são relevantes, facilitando a análise preditiva e o treinamento de modelos de machine learning.

Este modelo é a base fundamental dos Data Lakes e contrasta diretamente com o modelo schema-on-write (esquema na escrita) utilizado em Data Warehouses relacionais, onde as tabelas e campos devem ser criados e os dados limpos antes que qualquer informação possa ser armazenada.

2..2 MDW

O Modern Data Warehouse (MDW) é uma arquitetura híbrida que combina as capacidades de um Data Lake e de um Data Warehouse Relacional (RDW) lado a lado para criar um ambiente flexível e robusto.

Essa abordagem surgiu por volta de 2011, quando as empresas perceberam que os armazéns de dados relacionais sozinhos não conseguiam lidar com os desafios do Big Data, mas que os Data Lakes também não podiam substituí-los completamente.

A combinação funciona seguindo a lógica do "melhor de dois mundos":

- O papel do Data Lake: Ele é utilizado para a ingestão e o armazenamento de dados brutos em larga escala, servindo como uma área de preparação (staging) e refinamento.
  É o ambiente ideal para cientistas de dados explorarem dados brutos e treinarem modelos de machine learning.
- O papel do Data Warehouse Relacional (RDW): Ele é usado para o serviço de dados modelados, garantindo segurança, governança e conformidade.
  O RDW oferece alta performance e baixa latência para consultas complexas, sendo a ferramenta principal para usuários de negócios que consomem relatórios e painéis (dashboards).

.

Uma característica essencial dessa arquitetura é que ao menos parte dos dados deve ser replicada do Data Lake para o RDW; sem essa duplicação para fins de serviço e modelagem, a arquitetura seria classificada como um Data Lakehouse.

3. Data Lakehouse

O Data Lakehouse resolve os problemas históricos do Data Lake ao adicionar uma camada de software de armazenamento transacional sobre o repositório de dados existente. Essa arquitetura combina a flexibilidade e o baixo custo do Data Lake com o suporte a transações e a confiabilidade de um Data Warehouse (RDW).

Abaixo estão as principais formas como o Lakehouse resolve as limitações do Data Lake tradicional:

- Suporte a Comandos DML (UPDATE/DELETE): Diferente dos Data Lakes antigos, que exigiam a reescrita de arquivos inteiros para atualizações simples, o Lakehouse permite o uso de comandos como INSERT, UPDATE, DELETE e MERGE de forma eficiente.
- Transações ACID: O uso de camadas como Delta Lake, Iceberg ou Hudi garante as propriedades ACID (atomicidade, consistência, isolamento e durabilidade), o que assegura a integridade dos dados mesmo em casos de falhas do sistema.
- Prevenção de "Pântanos de Dados" (Data Swamps): Através do schema enforcement, o Lakehouse permite especificar e aplicar regras de esquema para os dados, rejeitando gravações que não estejam em conformidade e evitando a corrupção do repositório.
- Problema de "Pequenos Arquivos": O Lakehouse resolve a lentidão causada pelo excesso de arquivos pequenos utilizando algoritmos de compactação que unem esses arquivos automaticamente em arquivos maiores e mais eficientes.
- Melhoria de Performance: A arquitetura introduz recursos avançados como data skipping, indexação rápida e ordenação em Z (Z-order), que permitem pular dados irrelevantes e acelerar significativamente as consultas.
- Recurso de "Viagem no Tempo" (Time Travel): Graças aos logs de transação, é possível acessar versões anteriores dos dados para fins de auditoria, depuração ou recuperação de exclusões acidentais.
- Unificação de Lote e Streaming: O Lakehouse permite realizar processamento em lote (batch) e em tempo real (streaming) na mesma tabela, eliminando a necessidade de manter sistemas e pipelines separados para cada tipo de dado.

Enquanto o Data Lake original falha em substituir os Data Warehouses por ser complexo demais para usuários finais e carecer de governança, o Data Lakehouse

preenche essa lacuna fornecendo uma camada de serviço que faz o armazenamento de objetos parecer e funcionar como um banco de dados relacional.

3.1 Schema Enforcement no Data Lakehouse

O schema enforcement (ou aplicação de esquema) no Data Lakehouse é um recurso fundamental, geralmente associado a tecnologias como o Delta Lake, que permite especificar a estrutura esperada para os dados em uma tabela e aplicar regras para garantir que essas definições sejam seguidas.

As principais características e funções desse conceito, segundo as fontes, são:

- Garantia de Integridade: Ele assegura que todos os dados gravados em uma tabela estejam em conformidade com o esquema definido, aplicando regras como restrições de tipos de dados, restrições de nulidade (nullable constraints) e restrições de unicidade (unique constraints).
- Prevenção de Corrupção: Sem o schema enforcement, arquivos com esquemas inválidos ou incompatíveis poderiam ser adicionados a uma pasta de dados, o que frequentemente causa falhas em trabalhos de ELT e corrompe o repositório.
- Rejeição Automática: Se os dados de entrada não corresponderem exatamente ao esquema previsto, o sistema (como o Delta Lake) rejeita a operação de gravação e gera um erro, impedindo a entrada de dados "sujos" ou malformados.
- Evita o "Pântano de Dados" (Data Swamp): Ao aplicar essas regras de organização e governança, o schema enforcement ajuda a evitar que o Data Lake se torne uma coleção desorganizada e impossível de gerenciar, garantindo que os dados permaneçam utilizáveis para análise.

Em termos arquiteturais, esse recurso traz para o Data Lakehouse uma confiabilidade que anteriormente era exclusiva dos Data Warehouses relacionais (RDW), os quais operam sob o modelo de schema-on-write (esquema na escrita). Isso resolve um dos grandes problemas históricos dos Data Lakes tradicionais, que aceitavam qualquer arquivo sem validação prévia.

3.2 Diferença entre o MDW e o Data Lakehouse

A principal diferença entre o Modern Data Warehouse (MDW) e o Data Lakehouse reside na quantidade de repositórios de dados e na forma como as funcionalidades são integradas.

Enquanto o MDW é uma arquitetura híbrida que mantém um Data Lake e um Data Warehouse Relacional (RDW) operando lado a lado, o Data Lakehouse elimina a necessidade de um RDW separado, utilizando apenas o Data Lake para todas as finalidades.

Abaixo estão as distinções fundamentais baseadas nas fontes:

- Estrutura de Repositórios: No MDW, os dados são ingeridos no Data Lake para preparação e aprendizado de máquina e, em seguida, parte desses dados deve ser replicada para um RDW relacional para fins de visualização e relatórios de negócios.
  Já o Lakehouse utiliza um repositório único (o Data Lake), eliminando a duplicidade de sistemas.
- Camada Tecnológica: O Data Lakehouse só é possível graças a uma camada de software de armazenamento transacional (como Delta Lake, Apache Iceberg ou Hudi) que roda sobre o Data Lake, adicionando recursos antes exclusivos dos bancos de dados, como transações ACID e comandos DML (INSERT, UPDATE, DELETE).
- Confiabilidade e Atualização: O Lakehouse resolve o problema da latência de dados (staleness) encontrado no MDW, onde os dados no RDW são frequentemente mais antigos do que os do Data Lake devido aos ciclos de cópia.

  Além disso, evita falhas de sincronização entre os dois repositórios independentes do MDW.

- Custo Total de Propriedade (TCO): O Data Lakehouse geralmente oferece um custo menor, pois elimina o pagamento por dois tipos de armazenamento e evita o uso de motores computacionais relacionais que costumam ser mais caros que o processamento em Data Lake.
- Performance e Casos de Uso: Apesar da simplicidade do Lakehouse, o MDW ainda supera em performance de consultas relacionais complexas e dashboards que exigem latência de milissegundos, uma vez que os motores de RDW possuem otimizações (como indexação avançada) que o Data Lakehouse ainda luta para igualar completamente.
- Perfil do Usuário: No MDW, cientistas de dados acessam o Lake (dados brutos), enquanto usuários de negócios acessam o RDW (dados modelados).
  No Lakehouse, todos os usuários utilizam o mesmo repositório, embora os usuários de negócios muitas vezes precisem de uma camada de serviço relacional para facilitar a compreensão dos dados armazenados em arquivos.

O MDW foca, portanto, na coexistência de dois mundos para garantir performance e governança, enquanto o Data Lakehouse foca na unificação e simplificação da arquitetura através de tecnologia transacional sobre arquivos brutos.

Referência

SERRA, James. Deciphering Data Architectures: Choosing Between a Modern Data Warehouse, Data Fabric, Data Lakehouse, and Data Mesh. Sebastopol: O'Reilly Media, 2024.